DHARHSHNA U
231901008

CSE(CYBER SECURITY)

EXPERIMENT NO. 2

Developing and Deploying a Simple Chaincode

Aim

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Procedure / Observation

Step 1: Navigate to the Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

```
dharshna@DHARSHU:~$ cd ~/fabric-dev/fabric-samples/test-network
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

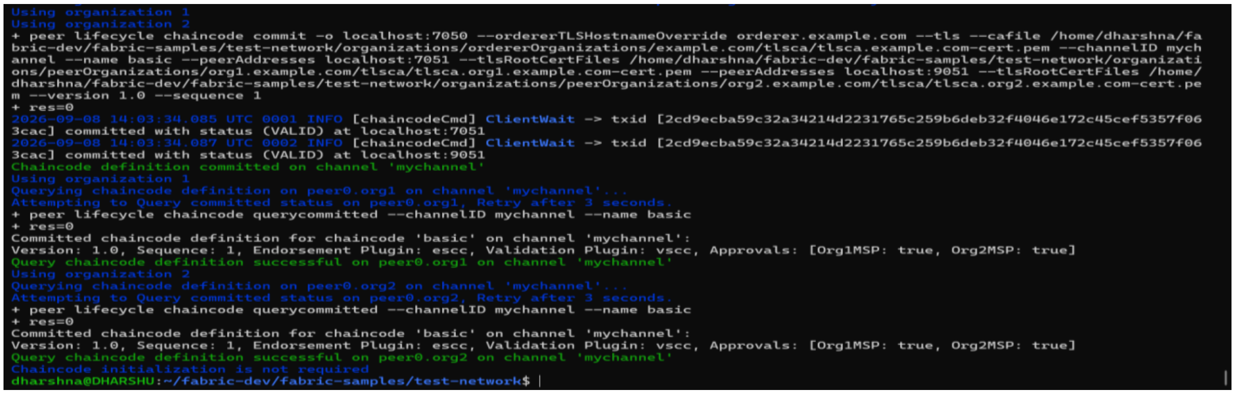
Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

```
pe.json --type common.Envelope --output /home/dharshna/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-09-08 13:58:21.594 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-09-08 13:58:21.609 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Step 3: Deploy the JavaScript Chaincode

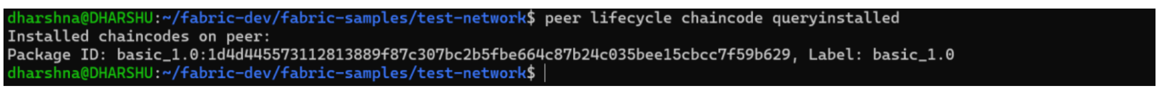
```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
```



```
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/dharshna/fa
bric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mych
annel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /home/dharshna/fabric-dev/fabric-samples/test-network/organizati
ons/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles /home/
dharshna/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pe
m --version 1.0 --sequence 1
+ res=0
2026-09-08 14:03:34.085 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [2cd9ecba59c32a34214d2231765c259b6deb32f4046e172c45cef5357f06
3cac] committed with status (VALID) at localhost:7051
2026-09-08 14:03:34.087 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [2cd9ecba59c32a34214d2231765c259b6deb32f4046e172c45cef5357f06
3cac] committed with status (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Step 4: Verify Installed Chaincode

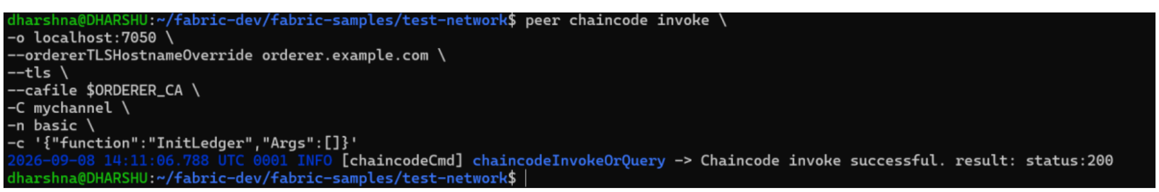
```
peer lifecycle chaincode queryinstalled
```



```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629, Label: basic_1.0
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Step 5: Invoke the Chaincode – Initialize Ledger

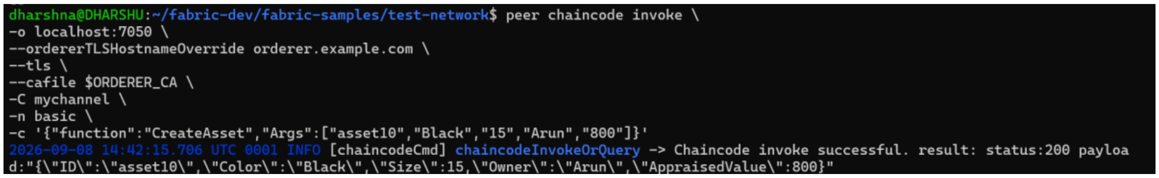
```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'
```



```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'
2026-09-08 14:11:06.788 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Step 6: Query the Ledger

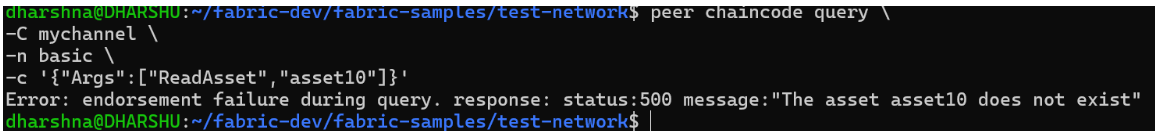
```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```



```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}'
2026-09-08 14:42:15.706 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payloa
d:"{\"ID\":\"asset10\",\"Color\":\"Black\",\"Size\":15,\"Owner\":\"Arun\",\"AppraisedValue\":800}"
```

Step 7: Create a New Asset

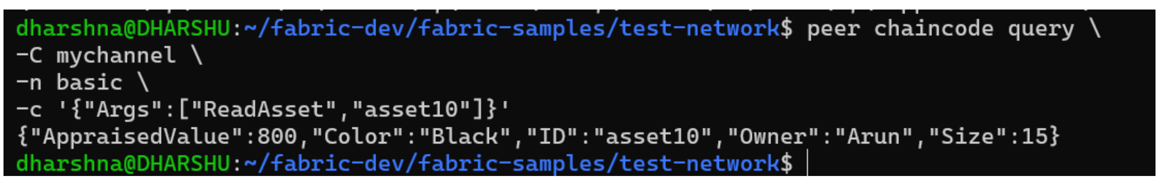
```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}'
```



```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset10 does not exist"
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Step 8: Query the Newly Created Asset

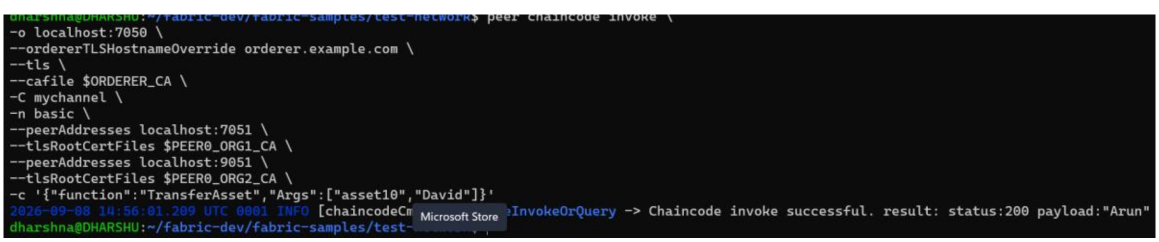
```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
```



```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":800,"Color":"Black","ID":"asset10","Owner":"Arun","Size":15}
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Step 9: Transfer Asset Ownership

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"TransferAsset","Args":["asset10","David"]}'
```



```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $PEER0_ORG1_CA \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $PEER0_ORG2_CA \
-c '{"function":"TransferAsset","Args":["asset10","David"]}'
2026-09-08 14:56:01.209 UTC 0001 INFO [chaincodeCm Microsoft Store InvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Arun"
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-
```

Step 10: Verify Updated Asset

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
```

```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":800,"Color":"Black","ID":"asset10","Owner":"David","Size":15}
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Step 11: Stop the Network

```
./network.sh down
```

```
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 10/10
 ✔Container peer0.org1.example.com      Removed                                  1.5s
 ✔Container ca_org1                     Removed                                 10.6s
 ✔Container ca_orderer                  Removed                                 10.9s
 ✔Container orderer.example.com         Removed                                  0.6s
 ✔Container ca_org2                     Removed                                 10.8s
 ✔Container peer0.org2.example.com      Removed                                  1.3s
 ✔Volume compose_peer0.org1.example.com Removed                                  0.1s
 ✔Volume compose_peer0.org2.example.com Removed                                  0.1s
 ✔Network fabric_test                   Removed                                  0.4s
 ✔Volume compose_orderer.example.com    Removed                                  0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bf
a908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:86e8e371dc78ebf968ff0d8f3c985841c63a9d5f6a593479ff0c5ee942f977f8
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaead
e1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:b0385cb2a347fbaf1553462288351439e8d55cf0ad266a8c0b44a980e76846ee
dharshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

Result

The simple JavaScript chaincode was successfully developed, deployed, installed, approved, committed, and executed on the Hyperledger Fabric test network. Asset records were created, queried, and transferred successfully, demonstrating the basic lifecycle of smart contracts in Hyperledger Fabric.